

AI 시대라는데, 그래서, 뭘 공부해야 한다구요?

당신이 알던 기본기는 죽었다.
지금 집중하고 10년을 앞서가기.

서문: 질문 자체가 함정이다

열심히 하고 있는데, 방향이 맞는지 모르겠습니다.

CS 기초가 중요하다는 말, 들었습니다. AI 도구를 익혀야 한다는 말도 들었습니다. 사이드 프로젝트, 도메인 심화. 다 해야 한다는 말을 들었고, 다 하려고 합니다.

그런데 어떤 날은, 이게 다 무슨 의미인지 모르겠습니다. 열심히 할수록 오히려 뒤흔치는 것 같습니다.

이상합니다.

두 사람이 같은 시점에 개발 공부를 시작했습니다. 둘 다 성실했습니다. 하나는 CS 기초부터, 다른 하나는 AI 도구부터. 둘 다 조언대로 했습니다.

6개월 뒤, 격차가 났습니다.

같은 조언, 같은 방식, 다른 결과.

지금까지 들어온 조언들을 한 군데 모아보면, 공통된 구조가 있습니다.

전부 “A냐 B냐”의 형태입니다. CS냐 AI냐. 알고리즘이냐 프로젝트냐.

이 구조 안에 숨겨진 전제가 있습니다.

“저 어딘가에 정답을 아는 사람이 있고, 내가 그것을 따라가면 된다.” 라는 전제입니다.

그런데,

정답을 아는 사람이 있을까요?

있다한들 그건 그 사람의 경로입니다. 우리가 듣는 조언은 전부 살아남은 사람들의 조언입니다. 살아남지 못한 사람들의 조언은 오지 않습니다.

즉, 생존 편향입니다.

지금 당장 문제에 직면한 우리에게 실제로 어떤 도움이 되나요 ?

앞서 언급한 두 사람은 격차를 보이기 시작합니다. 그런데 그 막막한 출발, 그 뿌리가 같았습니다.

Team Grit

1장: 모든 조언이 생존자 편향이다

사실, 이런 조언을 드리면 참 쉽습니다.

“CS 기초가 중요합니다. 자료구조, 알고리즘, 네트워크, OS — 이걸 오히려 더 중요해졌습니다.”

“AI 도구 활용 능력은 필수 스킬이 됐습니다.”

“시스템 설계와 도메인 이해에 비중을 높여야 합니다.”

“사이드 프로젝트를 처음부터 끝까지 만들어보세요.”

“특정 도메인을 깊이 파십시오.”

주마다 최소 2회 이상 1:1 멘토링 시션을 이어온지 벌써 3년이 다되어 갑니다.

시대마다 풀어야 하는 문제는 달랐고, 각각의 트렌드에서 나름의 어려움들은 있었지만 지금은 그 변동성과 혼란이 최고조입니다.

“지금 그래서 어떻게 해야 할까요?”

저 역시 회피하고 싶은 질문입니다. 그리고 저렇게 조언하면 저로서는 참 쉽습니다. 저런 말들, 얼마든지 멋지게, 근거 있게 말할 수도 있구요.

그런데 저 말들이 대체 무슨 도움이 되나요?

“기초 CS가 여전히 핵심이다”

이거 사실 개발자들이 10년째 하는 말이죠. “기초가 중요해”는 틀린 적이 없어서 반박이 안 되는 안전한 주장입니다. 그런데 문제는 어느 정도의 기초냐는 겁니다. 2026년 현실에서 AI가 코드 리뷰까지 해주고, “이 코드 리뷰 해줘”라고 하면 꽤 잘 합니다. “기초가 있어야 AI 결과물을 검증한다”고 했는데, AI가 검증도 해주는 시대가 오고 있습니다. 그러면 기초 CS 공부에 6개월에서 1년 투자하라는 조언이 정말 최적인지는 의문이 생깁니다.

“AI 도구 활용 능력은 필수 스킬이 됐다”

맞는 것 같으면서도 위험한 조언입니다. AI 도구는 6개월마다 판이 바뀝니다. 2024년에 이 도구가 대세였고, 2025년에 다른 도구가 치고 올라왔고, 지금도 계속 바뀌고 있습니다. “AI 도구를 잘 써라”는 말이 결국 특정 도구 숙련이 아니라 메타 능력을 키우라는 뜻인데, 그러면 너무 추상적이어서 신규 개발자 입장에서는 “그래서 뭘 하라고요?”가 됩니다. 그러니까 결국 그냥 겉도는 회피형 답변일 뿐인 거죠.

“시스템 설계와 도메인 이해에 비중을 높여라”

전형적인 현업자 편향의 생각입니다. 실제 트래픽, 실제 장애, 실제 트레이드오프 경험 없이 시스템 설계 영상을 보는 게 의미가 있을까요? 그리고 “AI가 시스템 설계는 못 한다”는 전제 자체가 얼마나 같지 모릅니다. 1년 전만 해도 “AI가 멀티파일 리팩토링은 못 한다”고 했는데 지금은 합니다.

이러한 조언은 어떤 의미에서는 스스로의 시야의 한계를 스스로 드러내는 말이기도 하고, 자부심 과잉에서 비롯된 권위 표출에 다름 아닙니다.

“사이드 프로젝트를 만들어라”

가장 클리셰한 조언이었습니다. 사이드 프로젝트의 문제는 대부분 사람들이 시작만 하고 완성을 못 한다는 겁니다. 그리고 AI 도구로 빠르게 뭔가를 만들 수 있게 되면서, 역설적으로 “만들어봤다”의 차별화 가치가 떨어졌습니다. “만들어봤다”가 아니라 “운영해봤다, 사용자를 받아봤다”까지 가야 의미가 있는데, 그건 또 다른 차원의 이야기입니다.

“도메인을 깊이 파라”

신규 진입자에게는 닭과 달걀 문제입니다. 도메인을 깊이 파려면 그 도메인에서 일해봐야 하고, 일하려면 먼저 취업을 해야 합니다. “결제 시스템을 이해하는 개발자가 되라”는 건 이미 그 도메인에 있는 사람한테나 의미 있는 조언이지, 진입 전인 사람한테는 “어느 도메인으로요?”라는 질문만 남깁니다.

결론적으로 이러한 조언들은 제가 멘토링 한 200명 이상의 신입, 그리고 1-2년 차 주니어분들께는 아무런 효력도 없는 말들입니다.

전부 이미 어느 정도 아는 사람에게만 유효할 뿐이죠.

조언하는 사람들이 전부 살아남은 사람들이기 때문입니다. 살아남지 못한 사람들의 조언은 우리에게 오지 않습니다. 개발자 커뮤니티 전체가 빠져 있는 함정입니다. 성공한 사람이 돌아보면서 “이래서 됐다”고 할 때, 그 말은 그 사람의 경로에서만 맞습니다. 같은 경로를 걷지 않은 사람에게는 다른 말이 됩니다.

많은 사람이 같은 말을 한다고 해서 그 말이 맞는 건 아닙니다.

《생각에 관한 생각》을 쓴 다니엘 카너먼은 “인지적 편의”라는 개념으로 이를 짚었습니다. 반복해서 들을수록 익숙해지고, 익숙한 것은 진실처럼 느껴집니다. 카너먼은 이 연구로 노벨 경제학상을 받았습니다.

돈이 중요한가요?

당연히 중요합니다.

그럼 돈이 최고의 가치인가요?

자본주의 시대 정신은 끊임없이 그렇게 속삭입니다. 대부분이 그 방향으로 행동합니다. 그렇다고 해서 그것이 진실인 건 아닙니다. 돈은 다른 가치를 교환하는 도구입니다. 목적이 되는 순간, 어딘가에서 어긋납니다.

개발자 커뮤니티도 다르지 않습니다.

“CS 기초”, “알고리즘”, “사이드 프로젝트”. 10년째 반복됩니다. 반복될수록 진실처럼 느껴집니다. 커뮤니티 전체가 같은 말을 한다고 해서, 그것이 당신에게 지금 필요한 것인지는 다른 문제입니다.

두 가지 전형이 있습니다

이 조언들을 성실하게 따르는 두 가지 전형이 있습니다.

굴방 개발자 — 혼자 깊이 팝니다. 커뮤니티를 끊고, 조용히, 자기만의 방식으로 기초를 쌓습니다. 결과물을 공개하지 않습니다. 완성될 때까지 기다립니다. 완성이 오지 않으면 기다림도 끝나지 않습니다.

수능식 개발자 — 암기로 증명하려 합니다. 알고리즘 100문제, CS 면접 질문 암기, 기술 스택 목록. 더 많이 알면 더 잘 될 거라는 믿음으로 계속 쌓습니다. 그런데 시장이 원하는 것은 지식의 양이 아닐 수 있습니다.

AI가 개인의 코딩 능력을 commodity로 만들기 시작한 지금, 두 전형 모두 시장이 원하는 것과 점점 어긋나고 있습니다.

그러면 이 조언들을 다 지우고 나면 무엇이 남을까요?

데카르트의 코기토처럼 의심 불가능한 것이 하나라도 있을까요?

2장: 다 지우고 남는 것

아닌 것들을 다 지워봅시다.

“기초를 해라” → 어느 정도의 기초인지 대답 못 함.

“도구를 익혀라” → 도구가 계속 바뀌는데 대답 못 함.

“시스템 설계를 해라” → 경험 없이 어떻게 하는지 대답 못 함.

“만들고 싶은 거 정해라” → 뭘 만들 수 있는지 모르는데 대답 못 함.

전부 지웠습니다.

하나가 남습니다.

의심할 수 없는 것

“일단 뭐라도 만져봐야 합니다.”

이것만은 의심할 수가 없습니다.

일단 의심할 수 없는 데에서부터 기초를 쌓아 봅시다.

뭘 공부하든, 어떤 도구를 쓰든, 어떤 도메인이든 — 손으로 뭔가를 만져보지 않으면 아무 일도 안 일어납니다.

이건 AI 시대 이전에도 맞았고, 이후에도 맞을 겁니다.

“만져보라는 말조차 공허하지 않을까요?”

아니죠. 새로운 분야에 진입할 때 깔끔한 로드맵이 있을 거라는 기대 자체가 환상입니다.

그것은 마치 한 번도 들어가 본 적 없는 어두운 방 문턱에 서서, 누군가 완벽한 조명과 평면도를 쥐여주기 전까
지는 한 발짝도 떼지 않겠다고 고집부리는 것과 같습니다.

어두운 방에서 공간의 형태를 파악하는 유일한 방법은 일단 손을 뻗어 차가운 벽을 더듬고, 발을 내디뎠다가 의자 모서리에 부딪히는 고통을 겪는 것입니다.

지도는 그 불안정한 '더듬거림'과 '부딪힘'이 끝난 뒤에야 비로소 그려지는 결과물이지, 결코 시작할 때 주어지는 안내서가 아닙니다. 그 기대를 버리는 순간, "일단 만져봐야 한다"는 말이 공허하지 않다는 걸 알게 됩니다.

기본기가 죽었다는 말의 진짜 의미

“다 지우고 남는 게 만지는 것이라면, 기초는 필요 없다는 이야기입니까?”

기초만으로 충분하던 시대가 끝난 것입니다.

최소한의 기초 — 만지면서 “이게 왜 이렇게 동작하는가?”를 물을 수 있는 수준 — 는 만지는 과정에서 쌓입니다. 암기로 얻는 기초가 아니라, 실패하면서 묻는 기초입니다.

“기본기의 죽음”이 이 의미입니다.

옛 기본기가 죽은 게 아닙니다. 그것만으로 충분하던 시대가 죽은 겁니다.

for 루프를 빈 페이지에 짤 수 있는 사람이 귀하던 시대는 끝났습니다. AI가 그것보다 빠르게, 더 많이 짭니다. 하지만 그 코드가 왜 이 구조인지, 다른 선택지는 무엇이었는지, 이게 틀렸다면 어디서 틀렸는지를 물을 수 있는 사람은 AI가 만들지 못합니다.

그 능력은 어디서 옵니까?

그런데 무엇을 쌓는 겁니까?

만지는 행위가 다 같은 게 아닙니다. 같은 도구로 같은 문제를 푸는데, 어떤 사람은 6개월 뒤 그 코드를 설명하지 못하고, 어떤 사람은 선택하지 않은 대안이 왜 안 되는지를 얘기합니다.

쌓이는 것의 정체를 알아야 합니다.

3장: 새로운 기본기의 정체

어떻게 만집니까?

예전에는 시작에 조건이 있었습니다.

코드 한 줄을 짜려면 언어 문법을 알아야 했습니다. 뭔가를 만들려면 환경 설정, 라이브러리, 프레임워크 기초가 먼저였습니다. 지식이 시작 허가증이었습니다. 허가증 없이는 문 앞에 서지 못했습니다.

AI가 그 허가증을 없앴습니다.

“로그인 기능 만들고 싶다”고 말하면 코드가 나옵니다. 이전 세대가 6개월을 투자해야 닿을 수 있었던 출발선에, 지금은 오늘 바로 설 수 있습니다.

그렇다면 지식의 가치는 사라진 걸까요?

역할이 바뀌었습니다.

이전의 지식은 시작 조건이었습니다. 없으면 만들기 자체가 불가능했습니다.

지금의 지식은 판단 도구입니다. AI가 만들어준 것이 맞는지 틀린지를 가려내는 도구. 더 나은 선택지가 있는지를 물어볼 수 있는 기반. 코드가 왜 그 구조인지 이해하는 눈.

타이밍이 바뀐 겁니다. 지식을 먼저 쌓고 시작하는 게 아닙니다. 시작하면서 지식이 필요해지고, 그 필요가 공부를 만듭니다. 필요에서 생긴 지식이 암기로 생긴 지식보다 오래 갑니다.

허가증이 사라졌다면 — 무엇이 사람을 가릅니까?

두 사람이 같은 AI로 같은 문제를 30분 동안 푼다.

결과물은 비슷합니다. 코드도 나왔고, 기능도 됩니다. 화면에서 보면 차이가 없습니다.

6개월 뒤를 봅니다.

한 사람은 그 코드를 설명하지 못합니다. 왜 이 구조를 선택했는지 모릅니다. “AI가 짰어요”라고 합니다.

다른 사람은 선택하지 않은 대안이 왜 안 되는지를 얘기합니다. “처음엔 이 구조로 가려 했는데, 이 경우에 이 문제가 생겨서 바꿨습니다”라고 합니다.

같은 AI를 썼습니다. 같은 30분이었습니다. 무엇이 달랐을까요?

30분 동안 두 사람이 한 일이 달랐습니다

한 사람은 AI 출력을 받았습니다.

다른 사람은 AI 출력에 질문을 했습니다.

“왜 이 구조입니까?”, “다른 선택지는 뭐가 있을까요?”, “이게 틀렸다면 어디서 틀렸을까요?”

이 세 질문이 30분의 차이를 만들었습니다. 6개월 뒤 전혀 다른 사람을 만들었습니다.

쌓이는 것은 코드가 아닙니다

판단입니다.

내가 어떤 선택을 했고, 왜 그 선택을 했고, 그게 맞았는지 틀렸는지를 인식하는 근육입니다.

이것이 AI 시대의 새로운 기본기입니다.

옛 기초와 새로운 기초

옛 기초	새로운 기초
for 루프를 빈 페이지에 짜기	문제를 오해 없이 설명하는 언어 능력

옛 기초	새로운 기초
버블 정렬 외워서 구현하기	설계 선택지를 비교·판단하는 눈
정규식 외우기	AI 출력의 맛고 틀림을 구분하는 비평 능력

옛 기초는 암기하고 증명하는 것이었습니다. 새로운 기초는 판단하고 선택하는 것입니다.

후자는 전자를 외워서 얻어지는 근육이 아닙니다. AI와 함께 수백 번 판단을 반복하는 과정에서만 붙는 근육입니다.

이것이 당신을 막고 있는 이유

“뭘 공부해야 하나”는 질문은 대부분 옛 기초를 쌓으려는 시도입니다. 더 많이 알면 더 잘 될 거라는 믿음으로, 명시적이고 수치화 가능한 것들을 쌓으려 합니다.

그런데 시장이 원하는 것은 그 위에 올라오는 사유의 방식입니다. 문제를 어떻게 정의하는지. 틀렸을 때 어떻게 반응하는지. 왜 그렇게 짰는지 설명할 수 있는지.

이것들은 암기로 쌓이지 않습니다.

그렇다면 이 판단 근육, 시장은 지금 원하고 있을까요?

아니면 여전히 코드를 외워서 증명하는 사람을 원하고 있을까요?

4장: 시장이 이미 답했습니다

취준생이 준비하는 방식이 있습니다.

알고리즘 100문제. 기초 CS. Spring Boot 포트폴리오. 다 맞는 것처럼 느껴집니다. 시장이 원하는 것을 채우고 있다는 느낌입니다.

그런데 채용공고를 직접 보면 어떨까요?

제 말이 아닙니다. 채용공고가 말하고 있습니다.

대기업은 이미 AI를 기본값으로 봅니다

카카오페이 채용공고 우대사항입니다.

직무 소개

지원자격 (공통)

- Java/Kotlin 등 JVM 언어로 서버 로직을 개발하고, 프로덕션 장애를 직접 해결해본 경험이 필요해요.
- Spring Boot, Spring Cloud를 이용해 마이크로서비스 아키텍처를 설계하고, 장애 복구 시나리오를 구축해본 분을 환영해요.
- RDBMS 환경에서 쿼리 최적화나 인덱스 튜닝으로 성능을 개선해본 적이 있으면 좋아요.
- OOP, MSA 개념을 적극적으로 적용해 코드 품질과 유지보수성을 함께 높여본 역량이 있으면 좋아요.
- Docker, Kubernetes로 무중단 배포나 롤백을 진행하며 자원을 효율적으로 스케일링해본 일이 있으면 좋아요.
- AI 도구(코드 리뷰, 로그 자동 분석, 테스트 자동 생성 등)를 활용해 개발 공수를 줄여본 경험이 있으면 좋아요.

“AI 도구(코드 리뷰, 로그 자동 분석, 테스트 자동 생성 등)를 활용해 개발 공수를 줄여본 경험이 있으면 좋아요.”

“있으면 좋아요”라고 썼지만, 이건 사실상 “없으면 뒤흔친다”는 뜻입니다. 대기업이 우대사항에 넣었다는 건 조만간 자격요건으로 올라간다는 신호입니다.

네이버 N캘린더 팀 공고입니다.

Who We Are

- 캘린더, 알림, Keep, 마이 와 같은 네이버의 개인화된 도구 서비스를 운영 및 개발하는 조직입니다.
- 네이버의 에이전트 서비스 Agent N과 연계하여 네이버 회원을 위한 에이전트 시나리오를 제안하고 개발합니다.
- 안정적인 운영 및 신규 기능 개발을 위해 AI를 활용한 다양한 생산성 혁신을 시도하고 있습니다.
- 에이전트 시대에 중요성이 높아지고 있는 개인화 도구의 새로운 기술적·기능적 가능성을 MCP 인터페이스와 연계하여 탐색하고 실현할 수 있는 조직입니다.

“Claude Code, Cursor 등 AI 도구를 적극 활용하여 개발 생산성을 향상시킨 경험을 보유하신 분.”

구체적인 도구 이름이 채용공고에 등장하기 시작했습니다. “AI 관심 있는 분”이 아니라 “이 도구를 써서 성과를 낸 분”을 찾고 있습니다.

라인플러스는 한 단계 더 나갑니다.

우대사항

- 실제 서비스에 AI 기술을 적용한 경험이 있고 깊은 이해도를 가진 분
- LangChain, LangGraph 등 AI 프레임워크를 활용한 서비스 개발 경험
- Redis, MongoDB 등 NoSQL 데이터베이스를 활용한 대용량 서비스 운영 경험 있으신 분
- Kafka 및 Spring Cloud Data Flow를 이용한 실시간 데이터 스트리밍 및 처리 경험 있으신 분
- 글로벌 환경에서의 서비스 개발 및 운영 경험을 가지신 분
- DevOps 환경에 대한 이해와 Kubernetes, Docker 등 컨테이너 기술 활용 경험이 있으신 분
- 영어 또는 다국어 협업 환경에서의 업무 경험이 있으신 분

“실제 서비스에 AI 기술을 적용한 경험이 있고 깊은 이해도를 가진 분. Langchain, Langgraph 등 AI 프레임워크를 활용한 서비스 개발 경험.”

“써봤다” 수준이 아닙니다. 실제 서비스에 적용하고, AI 프레임워크를 다룰 줄 아는 사람을 원합니다.

유니콘은 AI Native를 선언했습니다

무신사는 2026년 초 아예 AI Native Engineering Track이라는 이름의 채용을 열었습니다.

무신사는

'이미 완성된 개발자'보다

'함께 성장할 수 있는 개발자'를 찾습니다.

AI로의 전환은 선택이 아닌 필수입니다.

기술은 빠르게 변하고, AI는 우리의 일하는 방식을 바꾸고 있습니다.

이 변화 속에서 우리가 찾는 사람은 다음과 같습니다.

- 새로운 기술에 호기심을 가지고 빠르게 학습하는 분
- AI 도구를 활용해 더 나은 코드를 작성하고 싶은 분
- 완벽한 실력보다 함께 성장하려는 태도를 가진 분
- 실제 서비스를 만들며 사용자에게 가치를 전달하고 싶은 분

코딩테스트도 두 번입니다. 기본 코딩 역량 한 번, AI Agent 활용 능력 한 번. AI를 쓰는 능력 자체를 평가하겠다는 선언입니다.

컬리는 더 직접적입니다.

주요 업무

- Payment 및 Billing 시스템 설계/개발/운영
- 정산 플랫폼 및 회계 연동 시스템 설계/개발/운영
- 캐시 및 적립금(포인트) 시스템 설계/개발/운영
- AML(Anti-Money Laundering, 자금세탁방지) 시스템 개발/운영 및 컴플라이언스 대응
- AI를 활용한 신기술 및 업무효율을 위한 연구/개발
- 신규 프로젝트 PM/PL (Project Manager/Project Leader) 역할 수행

자격 요건

- 백엔드 개발(Java/Kotlin, Spring Framework) 경력 5년 이상인 분
- AWS 클라우드 및 MSA 환경 아키텍처 설계/운영 경험을 보유한 분
- 결제, 정산, 회계, 선불 등 핀테크 시스템 개발/운영 실무 경험을 보유한 분
- SAP, ERP 등 재무/회계 시스템 및 외부 금융 솔루션 API 연동 경험을 보유한 분
- AI Agent, MCP (Multi-Party Computation) 서버 연동 및 활용 경험을 보유한 분

우대 사항

- AI 기술(LLM 등)에 대한 높은 관심 및 업무 활용 능력을 보유한 분
- 선불카드, 포인트 등 도메인 지식 및 관련 법규에 대한 이해가 있는 분

“생성형 AI를 활용하는 과제전형(사전과제 또는 코딩테스트)이 진행됩니다.”

AI 없이 코딩하는 시대는 끝났다고 공식적으로 선언한 겁니다.

오늘의집은 이렇게 씁니다.

‘당연한 불편’을 없애고 새로운 기준을 세우는 일은 쉽지 않지만, 그래서 더 보람 있는 일입니다. Interior & Life 서비스팀은 실패를 두려워하지 않고, 기존의 당연함에 질문을 던지며, 업 자체를 바꾸는 경험을 함께할 도전적인 동료를 기다립니다. 또한, 오늘의집은 AI Native 조직으로 변화 중입니다. 우리는 AI를 도구처럼 쓰는 것을 넘어, 일의 방식 자체를 함께 바뀌나갈 동료를 기다리고 있습니다.

 AI와 동료처럼 함께 일하고 싶은 개발자라면, 바로 오늘의집이 당신의 Playground 입니다.

“GPT, Claude 등 LLM 기반 도구를 실제 업무에 적용해보고, 그 과정에서 얻은 인사이트나 한계를 설명할 수 있는 분.”

핵심은 “설명할 수 있는”입니다. 써봤다는 건 누구나 할 수 있습니다. 왜 쓰고, 어디서 한계를 느끼고, 어떻게 보완했는지를 자기 말로 설명할 수 있느냐 — 그게 차이입니다.

기존 역량은 이미 기본값이 됐습니다.

시장이 이미 질문을 바꿨습니다. 당신은 아직 예전 질문의 답을 준비하고 있었습니다.

두 지원자의 차이

자기중심적 어필이 있습니다.

“저는 CS 기초가 탄탄합니다. 알고리즘 100문제 풀었습니다. Spring Boot 포트폴리오가 있습니다.”

시장중심적 어필이 있습니다.

AI로 실제 사용자가 있는 것을 만들었고, 운영해봤고, 그 과정에서 어떤 판단을 내렸다는 증거가 있습니다.

정답은 문제에 있습니다.

출제한 사람이 채점합니다. 문제를 모르면 정답을 쓸 수가 없습니다.

그런데 대부분이 거꾸로 합니다.

“CS 기초가 탄탄합니다. 알고리즘 100문제 풀었습니다.” 교수님이 말했고, 커뮤니티가 말했고, 선배가 말했습니다. 그래서 그 답을 씁니다.

채점자가 그걸 묻지 않았는데.

채용공고가 묻는 것은 “AI로 실제 사용자가 있는 것을 만들어봤느냐, 그 과정에서 어떤 판단을 내렸느냐”입니다. 그 질문에 “알고리즘 100문제”를 답으로 제출하면 — 당연히 점수가 없습니다.

이것이 서류에서 계속 탈락하는 이유입니다.

멘토링 현장에서 보이는 편차

그 편차가 지금 멘토링 현장에서 극명하게 나뉘고 있습니다.

AI를 써서 실제로 운영해본 경험이 있는 분들은 일주일에 면접을 여러 개 봅니다. 그렇지 않은 분들은 서류에서 막힙니다.

같은 시기에 시작했고, 비슷한 노력을 했습니다. 그런데 결과가 이렇게 갈립니다.

차이는 무엇을 쌓았느냐입니다.

그러면 AI를 어떻게 써야 시장이 원하는 그 사람이 될까요?

단순히 코드를 뽑는 도구로 쓰는 것과, 판단 근육을 쌓는 방식으로 쓰는 것은 — 6개월 뒤 완전히 다른 사람을 만듭니다.

5장: 엔진의 진짜 용도

AI를 쓰면 빨라집니다.

기대가 생깁니다. AI를 많이 쓸수록 빨리 성장해야 합니다.

그런데 실제로는 그렇지 않습니다.

같은 AI를 6개월 쓴 두 사람

한 사람은 코드를 설명하지 못합니다. “AI가 짰어요.” 왜 이 구조인지 모릅니다. 다른 방식이 있는지도 모릅니다.

다른 사람은 그 코드가 왜 틀렸는지를 얘기합니다. “이 설계는 이런 경우에 무너집니다. 처음엔 다른 구조를 시도했는데 이 이유로 안 됐습니다.”

같은 AI였습니다. 같은 6개월이었습니다. 왜 이렇게 달라졌을까요?

한 사람은 output을 뽑았습니다.

다른 사람은 실패를 뽑았습니다.

잘못된 설계를 AI에게 먼저 만들게 하고, 그게 왜 무너지는지를 직접 봤습니다. “이 구조로 가면 어떻게 될까요?” 물었고, 망하는 걸 눈으로 확인했습니다. 그 실패가 판단 근육이 됐습니다.

변명의 목록

“시간이 없어서요.”

AI가 그 시간을 만들었습니다. 코드 작성 시간이 줄었습니다. 그 시간으로 무엇을 했는지가 6개월 뒤의 차이입니다.

“설계를 어떻게 해야 할지 모르겠어요.”

AI에게 세 가지 안을 받아서 고를 수 있습니다. 그 선택의 이유를 적으면, 그게 판단 기록이 됩니다.

“아직 실력이 부족한 것 같아서 준비를 더 하고요.”

준비가 끝나는 날은 오지 않습니다. 만지면서 느끼는 부족함이 가장 빠른 성장의 신호입니다.

오답 루프

3년 걸릴 실패를 3개월에 만나는 것.

이것이 엔진의 진짜 용도입니다.

빠르게 성공하기 위한 도구가 아닙니다. 빠르게 실패하기 위한 도구입니다. 실패의 속도가 빨라지면, 배움의 속도도 빨라집니다.

AI 없이 3년 동안 만날 실패를 3개월에 만났다면 — 그 사람은 3년치 판단을 3개월에 쌓은 겁니다.

Grit의 정의

편집증적 낙관주의자.

실패가 올 것을 알면서도, 그 실패가 결국 자신을 더 강하게 만든다고 믿는 사람입니다. 변명 없이 실패를 반복하는 능력입니다.

AI 시대에 가장 필요한 것은 AI를 잘 쓰는 능력이 아닙니다. AI로 더 빠르게 실패하고, 그 실패에서 판단을 뽑아내는 능력입니다.

그 사람들이 실제로 어떤 모습인지, 다음 장이 현장을 보여줍니다.

Team Grit

6장: 현장 패턴

판단을 쌓는 사람, 실패를 반복하는 사람 — 그 사람들이 실제로 어떤 모습인지, 요즘 우리가 자주 목격하는 네 가지 장면이 있습니다. 거창한 성공담이 아니라, 조심스러운 관찰입니다.

정말로 이렇게 사는 사람이 있을까요?

패턴 #1. “거절이 공부였다”고 말하는 신입들

첫 취업 초기에 PR이 반복해서 반려되는 경험을 겪는 분들이 있습니다. “AI가 뺀 걸 그냥 쓰면 반려된다”는 원칙을 가진 팀에 합류한 경우가 많습니다.

이 네 명 모두 처음부터 잘한 게 아닙니다.

리포트 #1. 3년 차가 된 1년 차, 서지우

서지우 씨는 2024년 3월에 입사한 신입 개발자입니다.

입사 첫 주, 팀장이 건넨 말은 이것이었습니다.

‘얼마나 많이 쓰느냐’보다 ‘얼마나 자주 피드백 루프에 올라타느냐’가 성장의 밀도를 결정한다는 관찰입니다.

지우 씨는 처음에 당황했습니다. “AI를 쓰되, 의심하라”는 말의 의미를 몰랐습니다. 첫 달, 그냥 AI가 준 코드를 가져다 냈습니다.

결과: 첫 달에 PR 56건 올렸습니다. 이 중 머지된 것은 9건. 나머지는 모두 반려, 재작업, 폐기.

거절 하나하나가 데이터였습니다. “AI가 왜 이걸 이렇게 짤까”, “팀장이 왜 이걸 반려했을까”를 47번 반복하면서, 지우 씨는 코드를 ‘읽는 법’을 익혔습니다.

6개월 뒤, 지우 씨는 팀의 주니어 2명을 멘토링하고 있었습니다. 1년 뒤, 사이드 프로젝트로 MRR(월 반복 매출) 300만 원짜리 SaaS를 혼자 올렸습니다.

“회사 월급이 주는 안정감과, 제가 만든 작은 시장에서 오는 안정감은 방향이 달라요. 숫자가 아니라 **중력의 방향**이 바뀌더라고요.”

다만 지우 씨도 한 번 멈췄던 적이 있습니다. PR 47번을 반려당하고 4개월 차에 “내가 못하는 게 맞나” 싶을 때, 팀장이 일대일로 코드 한 줄씩 같이 읽어준 30분이 있었다고 했습니다. “그 30분이 없었으면 저는 56건이 아니라 5건에서 멈췄을 거예요.” 거절을 데이터로 바꾸는 능력은 혼자 만들어지지 않았습니다.

‘회사에 의존하지 않는 채널’을 하나 만들어두는 습관이, 이제 훨씬 접근하기 쉬워졌습니다.

패턴 #2. AI 도입을 기다리다 결국 바깥으로 나간 중견들

연차가 쌓인 분들 중 한 갈래는, AI 도입이 더딘 회사에 있다가 답답함을 느껴 사이드 프로젝트 쪽으로 축을 옮겨가는 모습을 보입니다.

박현오 씨는 8년 차 백엔드 개발자였습니다. 대기업에서 안정적인 월급을 받았습니다.

그런데 2년 전부터 이상한 느낌이 들기 시작했습니다. 열심히 하는데, 성장하는 느낌이 없었습니다. 코드는 점점 잘 짜는데, 시장에서 자기 가치가 올라가는 느낌이 없었습니다.

그러다 2025년 1월, 그는 사표를 던졌습니다. 이유는 단 하나. “회사에서 AI 도입을 1년째 ‘검토 중’이라고 해서.”

퇴사 후 첫 달은 두려움이었습니다. 아무것도 없었습니다. 사수도, 팀도, 리뷰어도.

그는 부동산 소개 앱 하나를 **3주 만에** 런칭했습니다. Claude Code + Supabase + Vercel 조합. 프론트·백·디자인·배포·홍보를 전부 혼자 했습니다.

처음 두 달은 매출 거의 없었습니다. 그냥 만들고, 부수고, 다시 만들었습니다.

6개월 뒤 매출은 월 800만 원. 8년 차 연봉보다 많지는 않습니다. 아직은요. 하지만 그의 말은 인상적이었습니다.

퇴사 후 첫 두 달이 지옥이었던 이유는 매출이 아니었습니다. “내가 가는 방향이 맞는지 물어볼 사람이 없다”는 것이었습니다. 3개월 차에 사이드 프로젝트 하는 8년 차들이 모인 작은 슬랙 한 곳에 들어갔고, 거기서 매주 자기 시도를 공유하면서야 부수고 다시 만드는 사이클이 생산적으로 돌기 시작했습니다. “혼자 만든 건 맞는데, 혼자 판단하진 않았어요.”

“AI 시대의 리더십은 ‘가장 잘 짜는 사람’이 아니라, ‘**가장 잘 의심하는 문화를 설계하는 사람**’ 쪽으로 무게 중심이 이동하고 있습니다.”

패턴 #3. 팀 전체의 워크플로우를 바꾸는 설계자가 된 중간 연차들

2024년 초, 하늘 씨는 AI를 쓰기 시작했습니다. 처음에는 단순히 코드를 빠르게 짜는 용도였습니다. 한 달 지나니까, 그 시간으로 다른 걸 하고 싶어졌습니다.

PR 자동 리뷰, 테스트 자동 생성, 디자인 토큰 자동 동기화. 팀 전체가 쓸 수 있는 워크플로우를 혼자 설계했습니다. 이 하나로 팀 전체 릴리즈 속도가 2.4배 빨라졌습니다.

하늘 씨가 이 워크플로우 설계까지 갈 수 있었던 분기점은, 시니어 한 명이 “그거 너 혼자 쓸 거야, 팀 거로 만들 거야?”라고 물어본 한 마디였다고 합니다. 개인 무기로 그치려던 것을 팀 시스템으로 끌어올린 건 그 질문이었지, 그의 코딩 실력이 아니었습니다.

“저는 개발자라기보다 문제를 푸는 사람이에요. AI가 코드를 써주니까, 저는 문제만 잘 고르면 되더라고요.”

“도구를 개인 무기로 쓰는 것”과 “팀의 시스템으로 설계하는 것”의 차이가 점점 선명해지고 있습니다.

패턴 #4. 편견이 없어서 더 빨리 시작하는 비전공자들

비전공자분들이 AI를 통해 작은 서비스부터 공개하는 사례가 눈에 띄게 늘고 있습니다.

처음 3개월은 솔직히 막막했습니다. 터미널이 뭔지도 몰랐습니다. AI에게 물어보면서 하나씩 익혔습니다.

- **1개월:** 터미널, 깃, 배포의 개념을 AI에게 물어가며 익힘.
- **2개월:** 첫 프로토타입. 헬스장 파트너 매칭 앱.
- **3개월:** 사용자 12명 확보. 8명에게 거절당함.
- **6개월:** 피봇 3회 후, ‘운동 습관 챌린지’ 앱으로 변경.
- **12개월:** DAU 4,200명, 유료 전환 8%, MRR 1,200만 원.

이 12개월 안에 가장 결정적이었던 건 6개월 차 피봇 결정이었습니다. 비전공자 본인은 “이 앱을 6개월 더 끌고 갈지 접을지” 판단할 기준이 없었다고 했습니다. 운동 커뮤니티 운영자 두 명에게 자기 사용자 12명 데이터를 들고 1시간씩 물어본 끝에 피봇 방향이 잡혔습니다. AI는 코드를 짜줬지만, “어느 문제를 풀지”는 사람이 같이 잡아줬습니다.

“문제 선택력 + 실행 속도” 라는 조합이 예전보다 더 강력한 무기가 됐습니다.

그것은 “AI가 내 자리를 빼앗을까?” 가 아니었습니다.

“AI가 내 힘을 몇 배로 키워준다면, 나는 그 힘으로 어떤 문제를 풀고 싶은가?”

네 명의 공통점

이 네 사람에게는 **한 가지 공통점**이 있습니다. 돈도, 스펙도, 재능도 아닙니다.

그리고 그것은 처음부터 있던 게 아닙니다. **실패를 반복하는 과정에서 생긴 것**입니다.

그리고 한 가지 더, 잘 보이지 않지만 네 사람 모두에게 있었던 것이 있습니다. **혼자 통과되지 않는 지점**입니다. 지우 씨에게는 30분의 코드 리딩, 현오 씨에게는 매주 시도를 공유하는 8년 차 슬랙, 하늘 씨에게는 시니어의 한 마디, 비전공자에게는 운동 커뮤니티 운영자 두 명. 이름은 멘토일 수도 동료일 수도 커뮤니티일 수도 있지만, 형태는 같습니다. 막힌 자리에서 누군가가 옆에 있었다는 것.

“AI가 내 자리를 빼앗을까?”

이 질문을 버리고 다른 질문으로 대체한 사람들입니다.

“AI가 내 힘을 100배로 키워주는데, 나는 그 100배로 무엇을 할 것인가?”

이 질문의 전환이, 네 사람의 커리어를 바꿨습니다.

그런데 한 가지가 더 있었습니다

질문의 전환만으로는 충분하지 않았습니다. 네 사람 모두 그 전환된 질문을 들고 갔을 때, 옆에서 같이 답을 더 들어준 누군가가 있었습니다. 당신의 리포트 #5도 마찬가지일 겁니다. 질문은 당신이 바꿉니다. 그러나 그 다음 한 줄을 같이 쓸 사람은 따로 필요합니다.

당신의 리포트 #5

당신이 지금 있는 자리, 그리고 1년 뒤 이 리포트에 추가하고 싶은 한 줄을 적어 보십시오.

- 리포트 #5. 당신의 이름: _____
- 현재 연차: _____
- 지금 가장 두려운 것: _____
- 3개월 뒤 내가 쓰고 싶은 한 줄: _____

앞의 네 명도 다 두려움이 있었습니다. 그 두려움을 없앤 게 아닙니다. 두려움을 가진 채로 움직인 겁니다.

당신이 이 책을 덮는 순간부터, 당신의 리포트는 쓰이기 시작합니다.

에필로그: 그래서 내일 아침에 뭘 커야 하는가

“뭘 공부해야 하나”는 질문을 버리십시오.

대신 이렇게 물으십시오.

“뭘 만들어볼까?”

이 두 글자의 차이가 전혀 다른 6개월을 만듭니다.

3개월짜리 실험

목적은 완성이 아닙니다. 만지면서 자기한테 맞는 게 보이게 되는 것입니다.

1주차: 아무거나 하나 따라 만드십시오.

진짜 아무거나입니다. 할 일 목록 앱이든, 날씨 앱이든, 계산기든. 지금 시점이면 TypeScript + Next.js가 가장 빠르게 눈에 보이는 결과물을 만들 수 있습니다. AI를 적극적으로 쓰십시오. 이해가 안 되는 부분이 나와도 일단 넘기십시오. 전체 그림을 먼저 보는 게 우선입니다.

2~4주차: 따라 만든 것을 하나 바꿔보십시오.

이 순간 진짜 막힘이 시작됩니다. 튜토리얼에는 없는 문제가 생깁니다. “아 이걸 하려면 DB가 필요하구나”, “상태 관리가 이래서 필요하구나”를 몸으로 느끼게 됩니다. 기초 CS가 필요한 이유를 이론이 아니라 필요에 의해 알게 되는 단계입니다.

1~3개월: 누군가 쓸 수 있는 것 하나를 배포까지 하십시오.

완성도 낮아도 됩니다. 여기서 처음으로 “사용자”라는 개념이 생기고, “서버가 왜 필요한지”, “데이터를 어디에 저장하는지” 같은 질문이 자연스럽게 나옵니다.

여기까지가 혼자 갈 수 있는 구간입니다.

1주차 따라 만들기, 2~4주차 변형, 1~3개월 배포까지는 AI와 인내심만 있으면 혼자 갑니다. 막혀도 검색이 풀립니다. 그러나 배포 직후 “이 다음에 뭘 만들지”, “내가 잘 가고 있는지”, “이 코드가 1년 뒤 시장에서 얼마짜리인지” — 이 세 가지는 혼자 판단되지 않습니다. 6장의 네 사람이 멈췄던 자리도 정확히 여기였습니다. 기술이 막힌 게 아니라, **방향 판단이 막힌 자리**.

대부분 1~2개월차에 막힙니다

기술적 막힘이 아닙니다. 방향의 막힘입니다.

두 전형이 모두 이 앞에서 멈춥니다. 골방 개발자는 결과물을 공개하지 않은 채 계속 내부로 향합니다. 수능식 개발자는 정해진 답이 없는 막힘 앞에서 멈춥니다.

그 막힘을 혼자 통과할 수 없습니다. 6장의 네 사람도 통과하지 못했습니다 — 다만 그들 옆에는 통과하게 해준 한 사람이 있었습니다.

두 가지 원칙

기록하십시오.

만든 것을 기록하는 행위 자체가 판단을 언어화하는 훈련입니다. GitHub에 올리든 블로그에 쓰든 — “이걸 만들다가 이런 문제를 만났고 이렇게 해결했다”를 남기십시오. 이게 나중에 포트폴리오가 되고, 취업할 때 쓸 수 있는 유일하게 진짜인 증거가 됩니다.

혼자 하지 마십시오.

판단 근육은 피드백 없이 쌓이지 않습니다. “뭘 물어봐야 하는지 모르는 문제”를 해결해 주는 건 AI가 아닙니다. 옆에서 “야 너 이거 해봤어?” 하는 사람입니다. 6장의 지우 씨, 현오 씨, 하늘 씨, 비전공자 — 네 명 모두 그런 사람을 곁에 두는 데 성공했기 때문에 통과했습니다.

멘토링 현장에서 본 장면 하나

처음 멘토링에서 “뭘 공부해야 하나”고 물었던 분이 있었습니다.

3일 뒤, 다시 메시지가 왔습니다.

“이 코드가 맞는지 틀린지를 판단하는 기준이 저한테 없다는 걸 처음 알았어요. 그 기준은 어디서 오는 건가요?”

달라진 것은 질문의 방향이었습니다.

처음 질문은 밖을 향했습니다. 정답을 달라는 요청이었습니다.

3일 뒤의 질문은 자기 안을 향했습니다. 내가 판단하지 못한다는 사실을 인식했습니다.

그 인식이 시작입니다. 그러나 시작 다음 한 발은 곁에 있는 누군가와 같이 디딤니다.

이 착각만 안 해도 절반은 갈 겁니다

이력서 100장을 보면 90장이 똑같습니다.

JWT, Redis 캐싱, Kafka. 좀 더 나가면 ELK, Outbox 패턴, 멍등성 처리.

“기술 스택이 많을수록 좋다”는 착각입니다.

숫자를 넣으면 나아질 거라고 생각해서 숫자를 넣습니다. “API 응답 속도 30% 개선.” 다른 멘토링에서 “객관적 지표로 써라”는 말을 들었기 때문입니다.

그 조언이 틀린 건 아닙니다. 문제는 모두가 그렇게 쓰니까 더 이상 차별화가 아니라는 겁니다.

멘토링이 끝날 때마다 제가 정말 자주 듣는 말이 있습니다.

“그릿님 멘토링은 비싸서 정말 고민했어요. 왜 비싼지 알겠네요.”

왜일까요? 제가 뭐 대단한 걸 주기 때문일까요? 당장 합격하는 합격 템플릿이라도 뿌리나요?

아뇨.

제가 하는 건 딱 하나입니다.

‘뭘 공부해야 하나’고 물어온 분한테, ‘왜 그게 중요하다고 생각하세요?’를 되묻는 것.

처음엔 다들 당황합니다. 대부분 답을 못 합니다.

그 공백이 출발점입니다.

기술이 문제가 아닙니다.

생각하는 방식이 문제입니다.

그리고 이 책에서도 그 비밀을 많이 풀었습니다.

Grit Moments

이 책이 다룬 모든 문제는 결국 하나로 끝납니다.

막히는 구간은 반드시 옵니다. 그 구간은 혼자 통과되지 않습니다. 6장 네 사람의 공통점이 그것이었습니다.

판단 근육은 피드백 없이 자라지 않습니다.

시장은 가혹해 보이지만, 동시에 지독하게 투명합니다.

나이, 전공, 배경보다 결국 하나만 남습니다.

이 사람이 문제를 책임지고 해결할 수 있는가.

AI 시대는 오히려 그 본질을 더 선명하게 드러냅니다. AI를 쓸 수 있는 사람은 많아졌지만, 방향을 제시하고 결과를 책임질 수 있는 사람은 그만큼 희귀해졌습니다. 지금 시장은 그런 사람을 찾고 있습니다.

도구와 프레임워크만 쫓아가면 우리는 영원히 대체될까 불안해야 합니다. 하지만 나 자신의 엔지니어링 '체급' 자체를 끌어올리면, 어떤 기술이 등장해도 두렵지 않습니다.

단순한 기술 스택이 아니라, 문제를 대하는 사유의 방식. 타이핑하는 손가락이 아니라, 주어진 도구를 이용해 언어나 환경의 제약 없이 어떻게든 해내고야 마는 집념과 실행력. 이것이야말로 어떤 시장에서도 살아남게 할 진짜 무기입니다.

그릿 모멘츠는 그 “옆에 있는 한 사람”을 12주 동안 제도화한 프로그램입니다.

대상: 1~5년 차 개발자 / 비전공 전향자 / 사이드 프로젝트가 배포 직후에 멈춘 분.

12주 동안 받는 것:

- 매주 1회 1:1 — 당신이 만든 것을 같이 읽고, 다음 한 주의 방향을 같이 정합니다. 검색으로 안 풀리는 “방향 판단”을 옆에서 같이 합니다.
- 선배 멘티들 — 같은 길을 먼저 걷고 있는 동료들의 매주 사유록과 시도 공유. 6장 박현오 씨가 들어갔던 그 슬랙의 자리입니다.
- 멘토 리뷰 — 코드 한 줄, 이력서 한 줄, PR 한 건 단위로 “이게 시장에서 얼마짜리인지” 판단해드립니다.

기간: 12주 (이 기간 동안 6장 사례들이 통과한 분기점을 같이 통과합니다).

모집: 상시 모집 (정원 제한 — 자리가 차면 대기).

지금 시작하려면 카카오톡 오픈채팅에서 **무료 커피챗**을 신청하십시오. 12주 로드맵을 같이 진단해드립니다.

[Grit Moments 자세히 보기 →](#)